

Теоретический конспект №7

Тема: Уравнение Беллмана — основа обучения с подкреплением

Связано с: [note 6.md](#) — V и Q функции · [note 4.md](#) — Value-Based методы

1. Интуиция: зачем нам уравнение Беллмана?

До этого мы ввели понятие **функции ценности** (value function):

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Эта функция показывает:

Какое ожидаемое вознаграждение получит агент, если начнёт в состоянии s и будет следовать политике π .

Проблема

Чтобы вычислить $V_{\pi}(s)$, нужно просуммировать **все будущие вознаграждения**. Это дорого: агенту пришлось бы “прожить” каждую возможную траекторию до конца игры.

2. Идея Беллмана: рекуррентное определение ценности

Ричард Беллман (1957) предложил блестящее упрощение:

«Ценность состояния равна немедленной награде плюс ценность следующего состояния.»

Формально

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1})]$$

Это и есть **уравнение Беллмана**. Оно превращает задачу суммирования бесконечной серии в **рекуррентное вычисление**, где каждое значение опирается на следующее.

Интуиция на примере “мышь и сыр”

- Агент (мышь) в состоянии s_t получает награду R_{t+1} за движение вправо.
- Следующее состояние s_{t+1} также имеет свою ценность $V(s_{t+1})$.
- Тогда ценность исходного состояния:

$$V(s_t) = R_{t+1} + \gamma V(s_{t+1})$$

Таким образом, если мы знаем ценности соседних состояний, можем обновлять каждое новое.

3. Ключевые параметры уравнения

Параметр	Что означает	Пример
R_{t+1}	немедленная награда	+1 за шаг вперёд
γ	коэффициент дисконтирования ($0 \leq \gamma \leq 1$)	0.99 — “думаем о будущем”, 0.1 — “живём моментом”
$V(s_{t+1})$	ценность следующего состояния	ожидаемое вознаграждение “там”

Если $\gamma = 1$

→ агент учитывает **все будущие награды** одинаково (долгосрочный планировщик).

Если $\gamma = 0$

→ агент смотрит **только на текущую награду** (жадный, короткозорый агент).

4. Аналогия с динамическим программированием (DP)

Беллман сформулировал идею **разделения задачи на подзадачи**:

Если знаешь, сколько стоит “шаг вперёд”, и сколько стоит быть “дальше”, можешь построить всю функцию ценности шаг за шагом.

5. Расширение на Q-функцию

Иногда нас интересует не просто “ценность состояния”, а “ценность действия в этом состоянии”:

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) \mid S_t = s, A_t = a].$$

Тогда **уравнение Беллмана для Q-функции** записывается так:

Ожидаемое уравнение Беллмана для Q_{π} также записывается через π и динамику среды:

$$Q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \sum_{a'} \pi(a' \mid s') Q_{\pi}(s', a').$$

А соответствующее уравнение оптимальности для оптимальной функции записывается так:

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \max_{a'} Q^*(s', a').$$

Последнее лежит в основе алгоритма Q-Learning.

6. Пример на числах

Представь простую сетку:

Состояние	Награда	Следующее состояние	$V(S_{t+1})$	γ	$V(S_t)$
S_1	+1	S_2	4	0.9	$1 + 0.9 \times 4 = 4.6$
S_2	+2	S_3	5	0.9	$2 + 0.9 \times 5 = 6.5$
S_3	+5	(цель)	0	0.9	$5 + 0.9 \times 0 = 5$

7. Главное, что нужно понять

1. **Беллман — это не формула, а принцип.** Любая RL-модель основана на идее “разбивки” ценности на текущую награду + будущее.
2. **Q-Learning — частный случай Беллмана.** Мы не знаем реальную функцию V , но аппроксимируем её по мере взаимодействия агента со средой.

3. **Без Беллмана нет RL.** Он связывает теорию (ожидания и вероятности) и практику (итеративное обновление в коде).

8. Ключевые формулы

Для V-функции

$$V_{\pi}(s) = \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V_{\pi}(s') \right]$$

Для Q-функции

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a')$$

9. Что дальше

Динамическое программирование на базе уравнений Беллмана приводит к алгоритмам Policy Iteration и Value Iteration (оценка политики $\leftrightarrow$ улучшение политики, либо итерация по оптимальности). Сводка с формулами — в файле `rl_notes_consolidated.md` (раздел 5).

Основано на:

- Sutton & Barto, *Reinforcement Learning: An Introduction* (2020)
- Bellman, R., *Dynamic Programming* (1957)
- Hugging Face Deep RL Course, Unit 2